

□ La Programmation Orientée Objet en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Pourquoi la POO ?

La programmation procédurale (fonctions + variables globales) fonctionne bien pour les petits scripts, mais devient vite ingérable sur des projets réels. La POO apporte une réponse structurée à ce problème.

Le problème sans POO :

```
# Style procédural : tout est dispersé, rien n'est regroupé
nom_employe = "Bechir"
salaire_employe = 3000
equipe_employe = 5
```

```
def afficher_employe(nom, salaire):
    print(f"{nom} gagne {salaire}")
```

```
def augmenter_salaire(salaire, pct):
    return salaire * (1 + pct / 100)
```

```
# Avec 50 employés : 150 variables, chaos garanti
```

- **problème** : les données (nom, salaire) et les comportements (afficher, augmenter) sont séparés —il n'y a aucun lien logique entre eux
- **risque** : modifier salaire_employe n'importe où dans le code, sans aucun contrôle
- **conclusion** : plus le programme grandit, plus c'est incontrôlable

La solution POO :

```
# Style POO : données + comportements regroupés dans un objet
```

```
class Employe:
    def __init__(self, nom: str, salaire: float):
        self.nom = nom
        self.salaire = salaire

    def afficher(self):
        print(f"{self.nom} gagne {self.salaire}")

    def augmenter(self, pct: float):
        self.salaire *= (1 + pct / 100)
```

```
# Tout est encapsulé, propre, réutilisable
```

```
e = Employe("Bechir", 5000)
```

```
e.augmenter(10)
```

```
e.afficher() # Bechir gagne 5500.0
```

- **class** : mot-clé qui définit un nouveau type personnalisé
- **self** : référence à l'objet courant —c'est lui qui "possède" les données
- **__init__** : constructeur —appelé automatiquement à la création de l'objet
- **encapsulation** : les données et les fonctions qui les manipulent sont réunies au même endroit

Les 5 bénéfices concrets de la POO :

Bénéfice	Ce que ça veut dire concrètement
Structurer un programme complexe	Chaque entité du monde réel devient une classe (Client, Commande, Produit)
Modéliser des entités réelles	Le code ressemble au problème qu'il résout
Réduire le couplage	Changer Employe n'impacte pas Commande
Améliorer la maintenabilité	On sait exactement où toucher quand un bug survient
Faciliter les tests	On teste chaque classe indépendamment

□ Exemple réel : boutique en ligne

Sans POO, une boutique avec 100 produits, 500 clients et 1000 commandes, c'est des milliers de variables et fonctions éparpillées. Avec la POO :

```
class Produit:
```

```
    def __init__(self, nom, prix, stock):
```

```
        self.nom = nom
```

```
        self.prix = prix
```

```
        self.stock = stock
```

```
    def est_disponible(self):
```

```
        return self.stock > 0
```

```
class Client:
```

```
    def __init__(self, nom, email):
```

```
        self.nom = nom
```

```
        self.email = email
```

```
        self.panier = []
```

```

def ajouter_au_panier(self, produit: Produit):
    if produit.est_disponible():
        self.panier.append(produit)
        print(f"👉 {produit.nom} ajouté au panier de {self.nom}")
    else:
        print(f"👉 {produit.nom} est en rupture de stock")

class Commande:
    def __init__(self, client: Client):
        self.client = client
        self.lignes = list(client.panier) # snapshot du panier

    def total(self):
        return sum(p.prix for p in self.lignes)

    def afficher_recap(self):
        print(f"\n👉 Commande de {self.client.nom}")
        for p in self.lignes:
            print(f"  - {p.nom} : {p.prix} TND")
        print(f"  TOTAL : {self.total()} TND")

# Simulation
clavier = Produit("Clavier", 99, stock=5)
souris = Produit("Souris", 45, stock=0)
bechir = Client("Bechir", "bechir@mail.com")

bechir.ajouter_au_panier(clavier) # 📦 ajouté
bechir.ajouter_au_panier(souris) # 📦 rupture

cmd = Commande(bechir)
cmd.afficher_recap()

```

- **Produit, Client, Commande** : chaque entité du monde réel devient une classe indépendante
- **est_disponible()** : la logique métier est portée par l'objet lui-même — pas par une fonction externe
- **ajouter_au_panier(produit: Produit)** : les **types hints** rendent les signatures lisibles et autodocumentées
- **composition naturelle** : Commande contient un Client, qui contient une liste de Produit — le code reflète la réalité

2 □ Classe vs Objet

Définition

La distinction classe/objet est la pierre angulaire de la POO. Une confusion fréquente chez les débutants.

- **Classe** → le **plan** de construction, le moule, le type —il n'existe qu'en mémoire de programme, pas comme donnée concrète
- **Objet (instance)** → l'**élément concret** fabriqué à partir du plan —c'est lui qui occupe de la mémoire avec de vraies valeurs
- **instanciation** → l'acte de créer un objet à partir d'une classe (`p1 = Personne(...)`)
- **attribut** → une variable appartenant à un objet (`self.nom`, `self.age`)
- **méthode** → une fonction appartenant à une classe, qui agit sur ses attributs

□ Analogie : `Personne` est le plan d'architecte. `p1 = Personne("Bechir", 40)` est la maison construite à partir de ce plan. On peut construire 1000 maisons différentes avec le même plan.

Exemple de base

```
class Personne:
    # ↑ mot-clé "class" + nom en PascalCase (convention Python)

    def __init__(self, nom: str, age: int):
        # ↑ constructeur — Python l'appelle automatiquement lors de Personne(...)
        # ↑ "self" = l'objet en cours de création
        self.nom = nom      # crée l'attribut "nom" sur l'objet
        self.age = age      # crée l'attribut "age" sur l'objet

    def afficher(self):
        # méthode d'instance : reçoit toujours "self" en premier paramètre
        print(f"Nom: {self.nom}, Age: {self.age}")

p1 = Personne("Bechir", 47)  # instanciation → __init__ est appelé
p2 = Personne("Sarrah", 35) # deuxième objet indépendant

p1.afficher()  # Nom: Bechir, Age: 47
p2.afficher()  # Nom: Sarrah, Age: 35

# Chaque objet a SES propres données
print(p1.nom)  # Bechir
print(p2.nom)  # Sarrah
```

- **p1 et p2** sont deux objets distincts —modifier p1.nom ne touche pas p2.nom
 - **self** est automatiquement fourni par Python —on ne le passe jamais manuellement lors de l'appel
 - **PascalCase** pour les noms de classes (Personne, CompteBancaire) — c'est la convention universelle en Python
-

□ Comprendre la notion de membres de classes

Comment savoir combien d'objets ont été créés issus d'une classe? :

Il faut définir un membre avec **cls** et non **self** comme il est le cas du `**_compteur**`

```
class Ticket:
    # attribut DE CLASSE : partagé par tous les objets
    _compteur = 0

    def __init__(self, description: str):
        Ticket._compteur += 1          # incrémenté à chaque création
        self.numero = Ticket._compteur # chaque ticket a un numéro unique
        self.description = description
        self.resolu = False

    def resoudre(self):
        self.resolu = True

    def __str__(self):
        statut = "❑ résolu" if self.resolu else "❑ en attente"
        return f"Ticket #{self.numero} | {self.description} | {statut}"

    @classmethod
    def total_tickets(cls):
        return cls._compteur

# Création de plusieurs instances
t1 = Ticket("Écran noir au démarrage")
t2 = Ticket("Imprimante ne répond pas")
t3 = Ticket("Mot de passe oublié")

t2.resoudre()

for t in [t1, t2, t3]:
    print(t)
```

```
print(f"\n📄 Total tickets créés : {Ticket.total_tickets()}")
```

Ticket #1 | Écran noir au démarrage | 📄 en attente

Ticket #2 | Imprimante ne répond pas | 📄 résolu

Ticket #3 | Mot de passe oublié | 📄 en attente

📄 Total tickets créés : 3

- **_compteur** = 0 : attribut de classe —une seule copie, partagée entre toutes les instances
 - **self.numero** = **Ticket._compteur** : chaque objet "capture" la valeur au moment de sa création → numéro unique garanti
 - **__str__** : méthode spéciale qui définit ce qu'affiche `print(objet)` — beaucoup plus propre qu'une méthode `afficher()` manuelle
 - **@classmethod** : méthode qui reçoit la **classe** (`cls`) au lieu de l'instance — parfaite pour accéder aux attributs de classe
-

📄 Modelisation des objets

Un objet peut recevoir un autre objet en paramètre. C'est la base de toute architecture POO réelle :

```
class Etudiant:
    def __init__(self, nom: str):
        self.nom = nom
        self.notes = []

    def ajouter_note(self, note: float):
        if 0 <= note <= 20:
            self.notes.append(note)

    def moyenne(self) -> float:
        if not self.notes:
            return 0.0
        return sum(self.notes) / len(self.notes)

    def __str__(self):
        return f"{self.nom} | moyenne : {self.moyenne():.2f}/20"
```

```
class Classe:
    def __init__(self, nom: str):
        self.nom = nom
        self.etudiants = []
```

```

def inscrire(self, etudiant: Etudiant):
    self.etudiants.append(etudiant)

def major(self) -> Etudiant:
    # retourne l'étudiant avec la meilleure moyenne
    return max(self.etudiants, key=lambda e: e.moyenne())

def afficher_classement(self):
    print(f"\n📊 Classement — {self.nom}")
    classement = sorted(self.etudiants, key=lambda e: e.moyenne(), reverse=True)
    for rang, e in enumerate(classement, start=1):
        print(f"  {rang}. {e.nom} | moyenne : {e.moyenne():.2f}/20")

# Création des étudiants
ali = Etudiant("Ali")
lina = Etudiant("Lina")
omar = Etudiant("Omar")

for note in [14, 17, 12]: ali.ajouter_note(note)
for note in [18, 19, 16]: lina.ajouter_note(note)
for note in [10, 13, 11]: omar.ajouter_note(note)

# Création de la classe et inscription
classe_info = Classe("Informatique L2")
for e in [ali, lina, omar]:
    classe_info.inscrire(e)

classe_info.afficher_classement()
print(f"\n🏆 Major de promotion : {classe_info.major().nom}")

📊 Classement — Informatique L2
1. Lina | moyenne : 17.67/20
2. Ali | moyenne : 14.33/20
3. Omar | moyenne : 11.33/20

🏆 Major de promotion : Lina

```

- **Classe contient des Etudiant** : c'est la **composition** —une relation "a-un" ou "contient"
- **inscrire(self, etudiant: Etudiant)** : le type hint Etudiant documente clairement ce qu'attend la méthode
- **lambda e: e.moyenne()** : on trie les objets par leur méthode —les objets se comportent comme des données riches
- **enumerate(classement, start=1)** : astuce Python pour obtenir le rang directement dans la boucle

□ À retenir pour la partie 1 :

- Une **classe** est un moule, un **objet** est ce qu'on fabrique avec
- `__init__` s'exécute automatiquement à chaque `Nom-Classe(...)`
- `self` est toujours le premier paramètre d'une méthode — Python le fournit tout seul
- Les objets peuvent **contenir** d'autres objets → c'est la composition
- `__str__` remplace avantageusement les méthodes `afficher()` manuelles

Je vais générer 20 questions d'entretien intelligentes sur la POO Python, directement en Markdown enrichi.

□ 20 Questions d'Entretien —POO en Python

1) `self` —pourquoi obligatoire, et que se passe-t-il sans lui ?

□ **Énoncé** : Expliquez le rôle de `self`. Que se passe-t-il si on l'oublie ?

□ **Piège fréquent** : Croire que `self` est un mot-clé réservé Python —ce n'est qu'une **convention**. On pourrait l'appeler `this` ou `moi`, ça fonctionnerait.

class Demo:

```
def afficher(self):  
    print(f"Je suis l'objet : {self}")  
  
def sans_self(): # ← oublié de self  
    print("méthode sans self")
```

```
d = Demo()  
d.afficher()           # ☑ fonctionne — Python passe d automatiquement
```

try:

```
    d.sans_self() # ☒ TypeError: sans_self() takes 0 positional arguments but 1 was g  
except TypeError as e:  
    print(f"Erreur : {e}")
```

Preuve que "self" n'est qu'une convention

class Bizarre:

```
def afficher(moi):           # "moi" à la place de "self" — valide !  
    print(f"moi = {moi}")
```



```
Bizarre().afficher() # ☑ fonctionne parfaitement
```

- **self** = référence à l'instance courante —Python la passe automatiquement comme 1er argument
 - **d.afficher()** est équivalent à **Demo.afficher(d)** —c'est exactement la même chose
 - L'erreur *"takes 0 positional arguments but 1 was given"* signifie toujours : **self manquant**
-

2) Attribut de classe vs attribut d'instance —le piège de la mutation

□ **Énoncé** : Quelle est la différence ? Montrez un cas où la confusion provoque un bug silencieux.

□ **Piège fréquent** : Modifier un attribut de classe via **self** au lieu de **Nom-Classe** —crée un attribut d'instance qui **masque** l'attribut de classe sans le modifier.

```
class Equipe:
    membres = [] # ← attribut de CLASSE (partagé)
    count = 0    # ← attribut de CLASSE

    def __init__(self, nom: str):
        self.nom = nom # ← attribut d'INSTANCE (propre à chaque objet)
        Equipe.count += 1

    def ajouter(self, membre: str):
        self.membres.append(membre) # ☑ bug : modifie la liste PARTAGÉE
```

Démonstration du bug

```
e1 = Equipe("Dev")
e2 = Equipe("QA")
```

```
e1.ajouter("Bechir")
e2.ajouter("Sarra")
```

```
print(e1.membres) # ['Bechir', 'Sarra'] ← bug ! Sarra est dans l'équipe Dev ?!
print(e2.membres) # ['Bechir', 'Sarra'] ← même liste partagée
```

```
print(Equipe.count) # 2 ← correct, count bien géré via Equipe.count
```

☑ VERSION CORRECTE : liste propre à chaque instance

```
class EquipeCorrecte:
```

```

count = 0

def __init__(self, nom: str):
    self.nom = nom
    self.membres = [] # ← créée dans __init__ = propre à chaque objet
    EquipeCorrecte.count += 1

def ajouter(self, membre: str):
    self.membres.append(membre)

e1 = EquipeCorrecte("Dev")
e2 = EquipeCorrecte("QA")
e1.ajouter("Bechir")
e2.ajouter("Sarrah")

print(e1.membres) # ['Bechir']
print(e2.membres) # ['Sarrah']

```

- **Attribut de classe** : défini dans le corps de la classe, **partagé** entre toutes les instances —utile pour des compteurs, des constantes
- **Attribut d'instance** : défini via `self.x = ...` dans `__init__`, **propre** à chaque objet
- Le piège : une liste ou un dict de classe est **muté** (pas réassigné) → toutes les instances voient la modification

3) `super()` —pourquoi pas `Parent.__init__(self, ...)` ?

□ **Énoncé** : Expliquez `super()` et pourquoi il est préférable à l'appel direct du parent en héritage multiple.

□ **Piège fréquent** : Appeler `A.__init__(self)` directement en héritage multiple provoque des initialisations en double ou manquantes.

```

class A:
    def __init__(self):
        print("Init A")

class B(A):
    def __init__(self):
        super().__init__() # suit le MRO — chaîne correcte
        print("Init B")

class C(A):
    def __init__(self):
        super().__init__()

```

```

        print("Init C")

class D(B, C):
    # héritage multiple
    def __init__(self):
        super().__init__() # déclenche toute la chaîne MRO
        print("Init D")

print("=== Avec super() ===")
d = D()
# Init A ← appelé UNE SEULE fois grâce au MRO
# Init C
# Init B
# Init D

print("\nMRO de D :", [cls.__name__ for cls in D.mro()])
# [D, B, C, A, object]

# ❗ VERSION CASSÉE sans super()
class B2(A):
    def __init__(self):
        A.__init__(self) # appel direct
        print("Init B2")

class C2(A):
    def __init__(self):
        A.__init__(self) # appel direct
        print("Init C2")

class D2(B2, C2):
    def __init__(self):
        B2.__init__(self) # appelle A.__init__ via B2
        C2.__init__(self) # appelle A.__init__ une 2ème fois !
        print("Init D2")

print("\n=== Sans super() ===")
d2 = D2()
# Init A ← appelé DEUX fois — bug silencieux
# Init B2
# Init A ← doublon !
# Init C2
# Init D2

```

- **MRO (Method Resolution Order)** : ordre dans lequel Python cherche les méthodes dans la hiérarchie — calculé par l'algorithme **C3 linearization**
- **super()** respecte le MRO et garantit que chaque classe de la hiérarchie est initialisée **exactement une fois**

- `super()` sans arguments fonctionne depuis Python 3 —plus besoin de `super(NomClasse, self)`

4) Attribut privé —est-il vraiment inaccessible ?

- **Énoncé** : `__attribut` est-il vraiment privé en Python ? Prouvez-le.
- **Piège fréquent** : Confondre **name mangling** (protection technique) avec de la vraie **encapsulation** comme en Java.

```
class Coffre:
    def __init__(self, code: int):
        self.visible = "Je suis public"
        self._discret = "Convention : ne pas toucher"
        self.__secret = code      # name mangling → _Coffre__secret

    def verifier(self, tentative: int) -> bool:
        return tentative == self.__secret

c = Coffre(1234)

# Accès public : normal
print(c.visible)

# Accès "protégé" : fonctionne, juste une convention
print(c._discret)

# Accès privé direct : erreur
try:
    print(c.__secret)
except AttributeError as e:
    print(f"Accès direct bloqué : {e}")

# Accès via name mangling : POSSIBLE (Python ne cache rien vraiment)
print(f"Contournement : {c._Coffre__secret}")    # 1234 — accessible !

# Vérification du vrai nom de l'attribut
print(vars(c))
# {'visible': ..., '_discret': ..., '_Coffre__secret': 1234}
```

- **_nom** : convention "protégé"—signale aux autres développeurs "ne pas utiliser directement", mais pas de blocage
- **__nom** : Python le renomme en `_NomClasse__nom` (**name mangling**) — évite les collisions en héritage, pas une vraie sécurité
- **vars(obj)** : affiche le `__dict__` de l'objet —révèle tous les vrais noms

d'attributs

- En Python, la philosophie est *"we're all consenting adults"*—la convention prime sur la restriction forcée

5) @classmethod vs @staticmethod —quand choisir lequel ?

□ **Énoncé** : Quelle différence entre les deux ? Donnez un cas d'usage concret pour chacun.

□ **Piège fréquent** : Utiliser @staticmethod là où @classmethod serait nécessaire pour supporter l'héritage.

```
class Animal:
    espece = "Animal générique"

    def __init__(self, nom: str, age: int):
        self.nom = nom
        self.age = age

    # @classmethod → reçoit la CLASSE (cls) — utile pour les factories et l'héritage
    @classmethod
    def depuis_texte(cls, texte: str):
        # "Rex:5" → Animal("Rex", 5)
        nom, age = texte.split(":")
        return cls(nom, int(age))  # cls = la classe appelante (pas forcément Animal)

    # @staticmethod → ni self ni cls — utilitaire pur, lié thématiquement à la classe
    @staticmethod
    def est_age_valide(age: int) -> bool:
        return 0 < age < 150

    def __str__(self):
        return f"{self.nom} ({self.age} ans)"

class Chien(Animal):
    espece = "Canis lupus familiaris"

    # Factory method : cls = Animal ici
    a = Animal.depuis_texte("Simba:3")
    print(a)  # Simba (3 ans)

    # Avantage de @classmethod : héritage respecté — cls = Chien !
    rex = Chien.depuis_texte("Rex:5")
    print(type(rex))  # <class '__main__.Chien'> ← pas Animal !
    print(rex)  # Rex (5 ans)
```

```
# Méthode statique : même résultat peu importe si appelée sur la classe ou l'instance
print(Animal.est_age_valide(5))      # True
print(Animal.est_age_valide(-1))     # False
print(a.est_age_valide(200))          # False — même résultat depuis l'instance
```

- **@classmethod** : reçoit cls (la classe) — essentiel pour les **factory methods** et quand on veut que les sous-classes fonctionnent correctement
- **@staticmethod** : ne reçoit ni self ni cls — c'est une fonction ordinaire logée dans la classe pour des raisons d'organisation
- Règle simple : si la méthode n'a pas besoin de l'instance ni de la classe → @staticmethod ; si elle doit fonctionner avec les sous-classes → @classmethod

6) __str__ vs __repr__ — la bonne pratique

□ **Énoncé** : Quelle est la différence ? Que retourne Python si seul __repr__ est défini ?

□ **Piège fréquent** : Implémenter __str__ sans __repr__, et se retrouver avec <__main__.X object> dans les logs et les listes.

```
class Produit:
    def __init__(self, nom: str, prix: float, stock: int):
        self.nom = nom
        self.prix = prix
        self.stock = stock

    def __str__(self) -> str:
        # Pour l'utilisateur final : lisible
        return f"{self.nom} — {self.prix} TND"

    def __repr__(self) -> str:
        # Pour le développeur : précis, reproductible
        return f"Produit(nom={self.nom!r}, prix={self.prix!r}, stock={self.stock!r})"
```

```
p = Produit("Clavier", 99.9, 10)
```

```
print(str(p))          # Clavier — 99.9 TND                ← __str__
print(repr(p))         # Produit(nom='Clavier', prix=99.9, stock=10) ← __repr__
print(p)               # Clavier — 99.9 TND                ← print() appelle __str__
```

Dans une liste : c'est __repr__ qui est utilisé !

```
catalogue = [Produit("Clavier", 99.9, 10), Produit("Souris", 45.0, 3)]
print(catalogue)
```

```
# [Produit(nom='Clavier'...), Produit(nom='Souris'...)] ← __repr__

# Règle de fallback : si __str__ absent, Python utilise __repr__
class SansStr:
    def __repr__(self):
        return "SansStr()"

s = SansStr()
print(s)      # SansStr() ← __repr__ utilisé comme fallback pour __str__
```

- **__str__** : appelé par `print()` et `str()` —doit être **lisible** pour un humain
- **__repr__** : appelé par `repr()`, dans les listes, les logs, le shell interactif —doit être **précis et reproductible** (idéalement, `eval(repr(obj)) == obj`)
- **!r** dans les f-strings : applique `repr()` à la valeur —met les guillemets autour des chaînes automatiquement
- Bonne pratique : **toujours implémenter __repr__** en priorité ; **__str__** est optionnel

7) @property —pourquoi c'est plus pythonique que get_/set_

□ **Énoncé** : Transformez un attribut public en propriété validée sans casser le code existant.

□ **Piège fréquent** : Créer un getter/setter style Java (`get_age()`, `set_age()`) alors que Python fournit `@property` pour garder la syntaxe `obj.age`.

```
class Employe:
    def __init__(self, nom: str, salaire: float):
        self.nom = nom
        self._salaire = salaire      # stockage interne avec underscore

    @property
    def salaire(self):
        """Lecture du salaire."""
        return self._salaire

    @salaire.setter
    def salaire(self, valeur: float):
        """Modification avec validation."""
        if valeur < 0:
            raise ValueError("Le salaire ne peut pas être négatif")
        if valeur > 100_000:
            raise ValueError("Salaire plafonné à 100 000 TND")
```

```

        self._salaire = valeur

    @salaire.deleter
    def salaire(self):
        """Suppression contrôlée."""
        print("🗑 Suppression du salaire")
        del self._salaire

e = Employe("Bechir", 3000)

# Syntaxe d'attribut — mais passe par le getter/setter
print(e.salaire)      # 3000 — appelle le @property getter
e.salaire = 3500      # appelle le setter — validation incluse
print(e.salaire)      # 3500

try:
    e.salaire = -500    # 🚫 déclenche ValueError
except ValueError as e_err:
    print(f"Erreur : {e_err}")

# Astuce : propriété calculée (pas de setter nécessaire)
class Rectangle:
    def __init__(self, largeur: float, hauteur: float):
        self.largeur = largeur
        self.hauteur = hauteur

    @property
    def aire(self) -> float:
        return self.largeur * self.hauteur    # calculée à la volée, pas stockée

r = Rectangle(4, 5)
print(r.aire)      # 20 — accès comme attribut, mais calculé dynamiquement
r.largeur = 10
print(r.aire)      # 50 — toujours à jour automatiquement

```

- **@property** : transforme une méthode en accès attribut — le code appelant reste `obj.salaire`, pas `obj.get_salaire()`
- **@salaire.setter** : intercepte les assignations `obj.salaire = x` — parfait pour la validation
- **propriété calculée** : `aire` n'est jamais stockée — recalculée à chaque accès → toujours cohérente avec `largeur` et `hauteur`
- Avantage clé : on peut commencer avec un attribut public simple, puis ajouter un `@property` plus tard **sans changer le code appelant**

8) Le piège des valeurs par défaut mutables

□ **Énoncé** : Pourquoi `def __init__(self, items=[])` est un bug classique ? Quelle est la bonne pratique ?

□ **Piège fréquent** : C'est l'un des bugs Python les plus célèbres —la liste est créée **une seule fois** à la définition de la classe, pas à chaque instanciation.

❌ VERSION BUGUÉE

```
class PanierBug:
    def __init__(self, items=[]):  # ← La liste [] est créée UNE SEULE FOIS
        self.items = items
```

```
p1 = PanierBug()
p2 = PanierBug()
```

```
p1.items.append("Clavier")
print("p1:", p1.items)  # ['Clavier']
print("p2:", p2.items)  # ['Clavier'] ← BUG : p2 est contaminé !
print(p1.items is p2.items)  # True — même objet en mémoire !
```

❌ VERSION CORRECTE : None comme sentinelle

```
class PanierCorrect:
    def __init__(self, items=None):
        self.items = items if items is not None else []
        # ou plus court :
        # self.items = list(items) if items else []
```

```
p3 = PanierCorrect()
p4 = PanierCorrect()
```

```
p3.items.append("Clavier")
print("p3:", p3.items)  # ['Clavier']
print("p4:", p4.items)  # [] ❌ indépendant
```

Même problème avec les dicts

```
class ConfigBug:
    def __init__(self, options={}):  # ← même piège
        self.options = options
```

```
class ConfigCorrecte:
    def __init__(self, options=None):
        self.options = options if options is not None else {}
```

- **valeur par défaut** : évaluée **une seule fois** au moment où Python parse la définition de la fonction —pas à chaque appel
- **None comme sentinelle** : pattern universel en Python pour éviter ce piège

avec tout objet mutable (liste, dict, set)

- La règle : **jamais de liste, dict ou set comme valeur par défaut** — toujours None + création dans le corps
 - Ce bug affecte aussi bien les méthodes ordinaires que `__init__`
-

9) `isinstance()` vs `type()` — lequel choisir et pourquoi ?

□ **Énoncé** : Dans quel cas `type(obj) == Classe` est-il une mauvaise pratique ?

□ **Piège fréquent** : Utiliser `type()` dans du code polymorphique — ça casse dès qu'on utilise l'héritage.

```
class Animal:
    def parler(self):
        return "..."
```

```
class Chien(Animal):
    def parler(self):
        return "Wouf"
```

```
class ChienPolicier(Chien):
    def parler(self):
        return "WOUF WOUF"
```

```
rex = ChienPolicier()
```

```
# type() : teste le type EXACT — trop strict
print(type(rex) == Chien)          # False ← problème : ChienPolicier EST un Chien
print(type(rex) == ChienPolicier) # True
```

```
# isinstance() : teste la hiérarchie complète — correct en POO
print(isinstance(rex, Chien))      # True
print(isinstance(rex, Animal))    # True
print(isinstance(rex, ChienPolicier)) # True
```

```
# Cas réel : fonction qui accepte tout type d'animal
```

```
def faire_parler(animal):
    # MAUVAISE approche : casse avec les sous-classes
    if type(animal) == Animal:
        print(animal.parler())
```

```
# BONNE approche : respecte le polymorphisme
if isinstance(animal, Animal):
    print(animal.parler())
```

```
faire_parler(rex)    # WOUF WOUF ?

# isinstance() accepte aussi un tuple de types
valeur = 42
print(isinstance(valeur, (int, float)))    # True — int OU float
```

- **type(obj)** : retourne le type **exact** —utile seulement quand on veut distinguer strictement Chien de ChienPolicier
- **isinstance(obj, Classe)** : vérifie si l'objet est une instance de Classe **ou de n'importe quelle sous-classe** —respecte l'héritage
- En POO, on code "contre l'interface" (la classe parent), pas contre l'implémentation → **isinstance** est presque toujours le bon choix
- **isinstance(val, (int, float))** : raccourci élégant pour tester plusieurs types à la fois

10) MRO et héritage multiple —comment Python résout les conflits

□ **Énoncé** : Avec l'héritage multiple, comment Python décide-t-il quelle méthode appeler ? Expliquez le MRO.

□ **Piège fréquent** : Supposer que l'ordre de résolution est aléatoire ou imprévisible —il est en réalité **déterministe** via l'algorithme C3.

```
class Vehicule:
    def demarrer(self):
        print("Vehicule démarre")

class Terrestre(Vehicule):
    def demarrer(self):
        print("Terrestre démarre (roues)")

class Aquatique(Vehicule):
    def demarrer(self):
        print("Aquatique démarre (hélice)")

class Amphibie(Terrestre, Aquatique):
    pass    # pas de demarrer() → Python cherche dans Le MRO

a = Amphibie()
a.demarrer()    # Terrestre démarre (roues) ← premier dans Le MRO

# Visualiser Le MRO complet
print("\nMRO d'Amphibie :")
```

```

for i, cls in enumerate(Amphibie.mro()):
    print(f" {i+1}. {cls.__name__}")
# 1. Amphibie
# 2. Terrestre ← trouvé ici, s'arrête
# 3. Aquatique
# 4. Vehicule
# 5. object

# Forcer l'appel d'une classe spécifique
class AmphibieForce(Terrestre, Aquatique):
    def demarrer(self):
        Aquatique.demarrer(self) # force l'appel aquatique
        print("Mode amphibie activé")

AmphibieForce().demarrer()

# Cas d'erreur : MRO impossible → Python refuse
class X: pass
class Y(X): pass

try:
    # Python refuse une hiérarchie incohérente
    exec("class Z(X, Y): pass") # ← Y déjà après X dans la chaîne
except TypeError as e:
    print(f"\nMRO impossible : {e}")

```

- **MRO (Method Resolution Order)** : ordre linéaire dans lequel Python parcourt les classes pour trouver une méthode
- **Algorithme C3** : garantit que les parents apparaissent **après** leurs enfants et que l'ordre de déclaration est respecté
- `Amphibie.mro()` ou `Amphibie.__mro__` : affiche la liste complète — toujours se terminer par `object`
- Python lève `TypeError` si le MRO est incohérent (hiérarchie contradictoire) — fail rapide, pas de comportement imprévisible

11) `__slots__` — optimisation mémoire et restriction d'attributs

□ **Énoncé** : À quoi sert `__slots__` ? Quand l'utiliser, et quels sont ses inconvénients ?

□ **Piège fréquent** : Utiliser `__slots__` et s'étonner que `__dict__` n'existe plus, ce qui casse certaines bibliothèques (sérrialisation, etc.).

```
import sys
```

```

# Sans __slots__ : chaque instance a un __dict__ (dictionnaire)
class PointNormal:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# Avec __slots__ : stockage optimisé, pas de __dict__
class PointSlots:
    __slots__ = ("x", "y") # liste exhaustive des attributs autorisés

    def __init__(self, x, y):
        self.x = x
        self.y = y

p_normal = PointNormal(1, 2)
p_slots = PointSlots(1, 2)

# Comparaison mémoire
print(f"Normal : {sys.getsizeof(p_normal)} octets + dict {sys.getsizeof(p_normal.__dict__)} octets")
print(f"Slots : {sys.getsizeof(p_slots)} octets (pas de __dict__)")

# __slots__ bloque la création d'attributs non déclarés
try:
    p_slots.z = 3 # AttributeError
except AttributeError as e:
    print(f"Attribut refusé : {e}")

# __dict__ n'existe plus
print(hasattr(p_normal, "__dict__")) # True
print(hasattr(p_slots, "__dict__")) # False ← attention aux outils qui s'appuient sur __dict__

# Impact réel : 1 million d'objets
points_normaux = [PointNormal(i, i) for i in range(1_000_000)]
points_slots = [PointSlots(i, i) for i in range(1_000_000)]

import tracemalloc
tracemalloc.start()
big_normal = [PointNormal(i, i) for i in range(100_000)]
_, peak_n = tracemalloc.get_traced_memory(); tracemalloc.stop()

tracemalloc.start()
big_slots = [PointSlots(i, i) for i in range(100_000)]
_, peak_s = tracemalloc.get_traced_memory(); tracemalloc.stop()

print(f"\n100 000 objets normaux : {peak_n / 1024:.0f} Ko")

```

```
print(f"100 000 objets slots : {peak_s / 1024:.0f} Ko")
```

- **__slots__** remplace le **__dict__** par un stockage fixe en C → gain mémoire significatif (souvent 40-50%) pour de grandes collections d'objets
- **inconvénient** : pas de **__dict__** → certains outils (pickle par défaut, copy, ORMs) peuvent se comporter différemment
- **à utiliser quand** : on crée des **millions** d'instances (points géographiques, pixels, lignes de log)
- **à éviter quand** : on a besoin de flexibilité (ajout dynamique d'attributs) ou d'interopérabilité avec des bibliothèques

12) Classes abstraites —ABC et @abstractmethod

□ **Énoncé** : Comment forcer les sous-classes à implémenter certaines méthodes ? Montrez un exemple métier.

□ **Piège fréquent** : Croire qu'une méthode avec pass dans le parent suffit à forcer l'implémentation —Python n'émet aucune erreur jusqu'à l'appel.

```
from abc import ABC, abstractmethod
```

```
# Contrat : toute méthode de paiement DOIT implémenter payer() et rembourser()
class MethodePaiement(ABC):

    @abstractmethod
    def payer(self, montant: float) -> bool:
        """Effectue Le paiement. Retourne True si succès."""
        ...

    @abstractmethod
    def rembourser(self, montant: float) -> bool:
        """Effectue Le remboursement."""
        ...

    # Méthode concrète : partagée par toutes les sous-classes
    def reçu(self, montant: float):
        print(f"Reçu de paiement : {montant} TND via {self.__class__.__name__}")

# Tentative d'instanciation directe → erreur immédiate
try:
    m = MethodePaiement()
except TypeError as e:
    print(f"Instanciation interdite : {e}")

# Sous-classe incomplète → erreur à l'instanciation
```

```

class CarteInComplete(MethodePaielement):
    def payer(self, montant):
        return True
    # rembourser() manquante !

try:
    c = CarteInComplete()
except TypeError as e:
    print(f"Méthode manquante : {e}")

# Implémentations complètes et correctes
class CarteBancaire(MethodePaielement):
    def __init__(self, numero: str):
        self.numero = numero

    def payer(self, montant: float) -> bool:
        print(f"Paielement carte {self.numero[-4:]} : {montant} TND")
        return True

    def rembourser(self, montant: float) -> bool:
        print(f"Remboursement carte : {montant} TND")
        return True

class Virement(MethodePaielement):
    def payer(self, montant: float) -> bool:
        print(f"Virement de {montant} TND")
        return True

    def rembourser(self, montant: float) -> bool:
        print(f"Contre-virement de {montant} TND")
        return True

# Code générique : fonctionne avec n'importe quelle méthode de paielement
def traiter_commande(paielement: MethodePaielement, montant: float):
    if paielement.payer(montant):
        paielement.recu(montant)

traiter_commande(CarteBancaire("1234567890123456"), 199.9)
traiter_commande(Virement(), 500.0)

```

- **ABC** (*Abstract Base Class*) : classe mère qui définit un **contrat** —ne peut pas être instanciée directement
- **@abstractmethod** : méthode que toute sous-classe **doit** implémenter — Python lève `TypeError` à l'instanciation si elle manque
- **méthodes concrètes dans l'ABC** : autorisées —permettent de partager du comportement commun tout en forçant certaines implémentations

- Différence avec une interface Java : en Python, une ABC peut avoir des méthodes concrètes et des attributs → c'est un hybride classe abstraite / interface

13) dataclass —éviter le boilerplate répétitif

□ **Énoncé** : Réécrivez une classe avec @dataclass. Quelles méthodes sont générées automatiquement ?

□ **Piège fréquent** : Oublier que @dataclass ne gère pas la validation — __post_init__ est là pour ça.

```
from dataclasses import dataclass, field
from typing import List
```

? VERSION VERBOSE (sans dataclass)

```
class ProduitManuel:
    def __init__(self, nom: str, prix: float, tags: list):
        self.nom = nom
        self.prix = prix
        self.tags = tags

    def __repr__(self):
        return f"ProduitManuel(nom={self.nom!r}, prix={self.prix!r}, tags={self.tags!r})"

    def __eq__(self, other):
        return self.nom == other.nom and self.prix == other.prix
```

? VERSION DATACLASS : 3x moins de code

```
@dataclass
class Produit:
    nom: str
    prix: float
    tags: List[str] = field(default_factory=list) # Liste vide par défaut (évite le p
    en_stock: bool = True
```

Validation dans __post_init__ : appelé après __init__ généré

```
def __post_init__(self):
    if self.prix < 0:
        raise ValueError(f"Prix invalide : {self.prix}")
    self.nom = self.nom.strip().capitalize() # normalisation automatique
```

Méthodes générées automatiquement : __init__, __repr__, __eq__

```
p1 = Produit("clavier", 99.9, ["informatique", "bureau"])
p2 = Produit("clavier", 99.9)
```



```

p3 = Produit("souris", 45.0)

print(p1)                    # Produit(nom='Clavier', prix=99.9, tags=[...], en_stock=...)
print(p1 == p2)              # True ← __eq__ généré compare tous les champs
print(p1 == p3)              # False

# dataclass frozen : objet immutable (hashable → utilisable comme clé de dict)
@dataclass(frozen=True)
class Point:
    x: float
    y: float

pt = Point(1.0, 2.0)
print(hash(pt))              # hashable ☑

try:
    pt.x = 5.0                # ☑ FrozenInstanceError
except Exception as e:
    print(f"Immutable : {e}")

• @dataclass génère automatiquement : __init__, __repr__, __eq__
  —et optionnellement __lt__, __hash__, __frozen__
• field(default_factory=list) : solution propre pour les valeurs par
  défaut mutables —équivalent du pattern None + création manuelle
• __post_init__ : appelé juste après le __init__ généré —idéal pour
  validation et normalisation
• frozen=True : rend l'objet immutable et hashable → peut être utilisé
  comme clé de dictionnaire ou dans un set

```

14) __call__ —rendre un objet callable comme une fonction

☐ **Énoncé** : À quoi sert __call__ ? Donnez un exemple concret où c'est préférable à une simple fonction.

☐ **Piège fréquent** : Confondre objet() (appelle __call__) et objet.methode() —les deux sont valides mais ont des usages différents.

Cas d'usage : pipeline de transformation de texte configurable

```

class Nettoyeur:
    def __init__(self, minuscules=True, strip=True, remplacements=None):
        self.minuscules = minuscules
        self.strip = strip
        self.remplacements = remplacements or {}

```

```

def __call__(self, texte: str) -> str:
    if self.strip:
        texte = texte.strip()
    if self.minuscules:
        texte = texte.lower()
    for ancien, nouveau in self.replacements.items():
        texte = texte.replace(ancien, nouveau)
    return texte

# L'objet "se souvient" de sa configuration entre les appels
nettoyeur = Nettoyeur(replacements={"é": "e", "à": "a"})

textes = [" Bonjour ", "PYTHON EST GÉNIAL ", "À bientôt"]
for t in textes:
    print(repr(nettoyeur(t)))    # appelé comme une fonction !
# 'bonjour'
# 'python est genial'
# 'a bientot'

# Vérification : est-il callable ?
print(callable(nettoyeur))    # True
print(callable(print))       # True
print(callable(42))           # False

# Autre exemple : accumulateur avec état
class Compteur:
    def __init__(self, debut=0):
        self._valeur = debut

    def __call__(self, increment=1):
        self._valeur += increment
        return self._valeur

c = Compteur(10)
print(c())    # 11
print(c())    # 12
print(c(5))   # 17

```

- **__call__** : permet d'utiliser un objet **comme une fonction** —nettoyeur(texte) au lieu de nettoyeur.transformer(texte)
- Avantage sur une simple fonction : l'objet **garde un état** entre les appels (configuration, compteur, cache...)
- **callable(obj)** : retourne True si l'objet possède **__call__** —bon moyen de vérifier avant d'appeler
- Très utilisé dans les frameworks ML (les modèles Keras/PyTorch sont appelables : modele(inputs))

15) `__new__` vs `__init__` —contrôle de la création d'objet

□ **Énoncé** : Quelle est la différence entre `__new__` et `__init__` ? Implémentez un Singleton avec `__new__`.

□ **Piège fréquent** : Implémenter un Singleton dans `__init__` —c'est trop tard, l'objet est déjà créé.

`__new__` : crée l'objet (appelé en 1er, retourne l'instance)
`__init__` : initialise l'objet déjà créé (appelé en 2nd, ne retourne rien)

```
class Trace:
    def __new__(cls, *args, **kwargs):
        print(f"1. __new__ : création de l'objet en mémoire")
        instance = super().__new__(cls)    # allocation réelle
        return instance                    # DOIT retourner l'instance

    def __init__(self, nom: str):
        print(f"2. __init__ : initialisation de {nom}")
        self.nom = nom
```

```
t = Trace("test")
# 1. __new__ : création de l'objet en mémoire
# 2. __init__ : initialisation de test
```

□ SINGLETON avec `__new__` : une seule instance possible

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Première création — instance unique")
            cls._instance = super().__new__(cls)
        else:
            print("Instance existante retournée")
        return cls._instance

    def __init__(self):
        # __init__ est appelé à chaque "création" — attention !
        pass
```

```
s1 = Singleton()    # Première création
s2 = Singleton()    # Instance existante
s3 = Singleton()    # Instance existante
```

```

print(s1 is s2)    # True ← même objet en mémoire
print(s1 is s3)    # True

# Cas d'usage réel : connexion à une base de données (une seule connexion)
class ConnexionDB(Singleton):
    def __init__(self):
        if not hasattr(self, "_connecte"):  # évite la réinitialisation
            self._connecte = True
            print("☑ Connexion DB établie")

db1 = ConnexionDB()  # Connexion DB établie
db2 = ConnexionDB()  # (rien — déjà initialisé)
print(db1 is db2)    # True

```

- **__new__** : méthode de **classe** (reçoit cls) —alloue et retourne le nouvel objet —appelée **avant** **__init__**
- **__init__** : méthode d'**instance** (reçoit self) —initialise l'objet déjà créé —ne retourne rien (None)
- Si **__new__** ne retourne pas une instance de cls, **__init__** n'est **pas** appelé
- **Singleton** : pattern qui garantit une unique instance —utile pour connexions DB, configurations globales, caches

16) shallow copy vs deep copy —piège en POO

☐ **Énoncé** : Pourquoi `b = list(a)` ne protège pas toujours contre les modifications ? Comment corriger ?

☐ **Piège fréquent** : Croire que copier une liste crée une copie indépendante de tout son contenu.

```

import copy

class Adresse:
    def __init__(self, ville: str):
        self.ville = ville

    def __repr__(self):
        return f"Adresse({self.ville!r})"

class Client:
    def __init__(self, nom: str, adresse: Adresse, historique: list):
        self.nom = nom
        self.adresse = adresse

```

```

        self.historique = historique

    def __repr__(self):
        return f"Client({self.nom!r}, {self.adresse}, historique={self.historique})"

# Objet original
original = Client("Bechir", Adresse("Tunis"), ["commande_1", "commande_2"])

# Shallow copy : copie l'objet, mais PAS les objets imbriqués
shallow = copy.copy(original)

# Deep copy : copie tout récursivement
deep = copy.deepcopy(original)

# Modification de l'adresse (objet imbriqué)
original.adresse.ville = "Sfax"
original.historique.append("commande_3")

print("Original :", original)
print("Shallow  :", shallow)    # adresse=Sfax et historique avec commande_3 — partagé
print("Deep    :", deep)       # adresse=Tunis et historique sans commande_3 — indépe

print("\nComparaison des identités :")
print(f"shallow.adresse is original.adresse : {shallow.adresse is original.adresse}")
print(f"deep.adresse   is original.adresse : {deep.adresse   is original.adresse}")



- shallow copy (copy.copy(), list(x), x[:]) : crée un nouvel objet mais ses attributs pointent vers les mêmes objets que l'original
- deep copy (copy.deepcopy()) : crée un arbre entier de copies — chaque objet imbriqué est dupliqué indépendamment
- Règle pratique : si l'objet ne contient que des types immuables (int, str, tuple), shallow suffit ; dès qu'il y a des listes, dicts ou autres objets → deep copy

```

17) Méthode liée vs non liée —comprendre le descripteur

□ **Énoncé** : Quelle différence entre instance.methode et Classe.methode ? Démontrez-le.

□ **Piège fréquent** : Stocker une méthode dans une variable et ne pas comprendre pourquoi elle "se souvient" de l'instance.

```

class Calculatrice:
    def __init__(self, valeur: float):
        self.valeur = valeur

```

```

def doubler(self):
    return self.valeur * 2

def ajouter(self, x: float):
    return self.valeur + x

c = Calculatrice(10)

# Méthode LIÉE : L'instance est déjà attachée
methode_liee = c.doubler
print(type(methode_liee))          # <class 'method'>
print(methode_liee.__self__)       # <__main__.Calculatrice object> ← instance attachée
print(methode_liee())              # 20 — pas besoin de passer l'instance

# Méthode NON LIÉE : simple fonction, l'instance doit être passée manuellement
methode_non_liee = Calculatrice.doubler
print(type(methode_non_liee))      # <class 'function'>
print(methode_non_liee(c))         # 20 — on passe c explicitement

# Application pratique : callbacks et délégation
class Bouton:
    def __init__(self, label: str, action):
        self.label = label
        self.action = action      # stocke une méthode liée

    def cliquer(self):
        print(f"[{self.label}] cliqué →", end=" ")
        self.action()

calc = Calculatrice(7)
btn = Bouton("Doubler", calc.doubler)  # méthode liée : calc "embarquée"
btn.cliquer()      # [Doubler] cliqué → 14

# La méthode liée se souvient de calc même après réassignation
calc.valeur = 100
btn.cliquer()      # [Doubler] cliqué → 200 ← suit les changements de calc

```

- **méthode liée** : accédée via `instance.methode` — l'instance est "attachée" dans l'objet méthode via `__self__`
- **méthode non liée** : accédée via `Classe.methode` — c'est une fonction ordinaire, `self` doit être fourni manuellement
- **descripteur** : le mécanisme Python qui transforme une fonction en méthode liée à l'accès — c'est `__get__` dans les coulisses
- Très utile pour les **callbacks** : passer `objet.methode` comme argument garantit que l'appel agira toujours sur le bon objet

18) Composition vs Héritage —quand choisir quoi ?

□ **Énoncé** : Expliquez le principe "Favor composition over inheritance". Donnez un exemple où l'héritage échoue.

□ **Piège fréquent** : Utiliser l'héritage pour réutiliser du code alors que la relation "est-un" n'est pas vraie.

☒ MAUVAIS DESIGN : héritage pour réutiliser Le code de sauvegarde

```
class BaseDeDonnees:
    def sauvegarder(self, data):
        print(f"Sauvegarde en DB : {data}")

class Rapport(BaseDeDonnees):    # Un Rapport EST-IL une BaseDeDonnees ? Non !
    def __init__(self, titre):
        self.titre = titre

    def generer(self):
        self.sauvegarder(self.titre)    # réutilise la méthode parent
```

Problème : Rapport hérite AUSSI de toutes Les méthodes DB non voulues

☑ BON DESIGN : composition — Rapport A-UN système de sauvegarde

```
class SauvegardeDB:
    def sauvegarder(self, data):
        print(f"☑ DB : {data}")

class SauvegardeFichier:
    def sauvegarder(self, data):
        print(f"☑ Fichier : {data}")

class SauvegardeCloud:
    def sauvegarder(self, data):
        print(f"☑ Cloud : {data}")

class RapportCompose:
    def __init__(self, titre: str, sauvegarde):
        self.titre = titre
        self._sauvegarde = sauvegarde    # injecté → interchangeable

    def generer(self):
        contenu = f"Rapport: {self.titre}"
        print(f"☑ Génération de '{self.titre}'")
        self._sauvegarde.sauvegarder(contenu)
```

```
# Flexibilité totale : changer le backend sans toucher à RapportCompose
r1 = RapportCompose("Ventes Q1", SauvegardeDB())
r2 = RapportCompose("RH 2024", SauvegardeFichier())
r3 = RapportCompose("Bilan", SauvegardeCloud())
```

```
for r in [r1, r2, r3]:
    r.generer()
    print()
```

```
# Règle mnémotechnique :
# "est-un" → héritage (Manager EST UN Employé)
# "a-un" → composition (Voiture A UN Moteur)
# "peut-faire" → mixin ou ABC
```

- **héritage** : modélise une relation "est-un"—Chien EST UN Animal, Manager EST UN Employé
- **composition** : modélise une relation "a-un"—Voiture A UN Moteur, Rapport A UN système de sauvegarde
- **avantage clé de la composition** : le comportement est **interchangeable à l'exécution** sans modifier la classe —c'est le principe ouvert/fermé (SOLID)
- **"Favor composition over inheritance"** : maxime du GoF (*Gang of Four*) —l'héritage crée un couplage fort et difficile à modifier

19) __enter__ / __exit__ —le protocole context manager

□ **Énoncé** : Comment implémenter with monobjet as x pour vos propres classes ?

□ **Piège fréquent** : Oublier de gérer les exceptions dans __exit__ —une ressource peut rester ouverte si l'exception n'est pas capturée.

```
class ConnexionFichier:
    def __init__(self, chemin: str, mode: str = "r"):
        self.chemin = chemin
        self.mode = mode
        self._fichier = None

    def __enter__(self):
        print(f"🔓 Ouverture de {self.chemin}")
        self._fichier = open(self.chemin, self.mode)
        return self._fichier # valeur de "as x" dans le with

    def __exit__(self, exc_type, exc_val, exc_tb):
```



```

        print(f"❏ Fermeture de {self.chemin}")
        if self._fichier:
            self._fichier.close()

        if exc_type is not None:
            print(f"❏ Exception interceptée : {exc_val}")
            return False      # False → propage l'exception ; True → La supprime

# Utilisation avec "with"
import tempfile, os

with tempfile.NamedTemporaryFile(mode="w", suffix=".txt", delete=False) as tmp:
    tmp.write("ligne 1\nligne 2\nligne 3")
    tmp_path = tmp.name

with ConnexionFichier(tmp_path) as f:
    for ligne in f:
        print(ligne.strip())
# ❏ Ouverture...
# ligne 1 / ligne 2 / ligne 3
# ❏ Fermeture... ← garanti même si exception

os.unlink(tmp_path)

# Version simplifiée avec @contextmanager (alternative sans classe)
from contextlib import contextmanager

@contextmanager
def chronometre(label: str):
    import time
    debut = time.time()
    print(f"❏ Début : {label}")
    try:
        yield                                # point d'entrée du bloc "with"
    finally:
        duree = time.time() - debut
        print(f"❏ Fin : {label} — {duree:.4f}s")

with chronometre("calcul intensif"):
    total = sum(range(10_000_000))

```

- **__enter__** : appelé au début du with —doit retourner la ressource (la valeur assignée à as x)
- **__exit__(exc_type, exc_val, exc_tb)** : appelé à la sortie du bloc —**toujours**, même si une exception se produit
- **retourner False** (ou None) dans **__exit__** : propage l'exception ; **re-**

tourner True : la supprime silencieusement

- **@contextmanager** : alternative légère pour les cas simples —yield délimite l'entrée et la sortie du contexte

20) Mixins —réutilisation horizontale sans héritage profond

□ **Énoncé** : Qu'est-ce qu'un Mixin ? Comment l'utiliser pour ajouter des comportements transversaux ?

□ **Piège fréquent** : Confondre Mixin et classe parent ordinaire —un Mixin ne doit **jamais** être instancié seul et ne doit pas avoir de `__init__`.

Mixins : petites classes de comportement, conçues pour être combinées

```
class JsonMixin:
    """Ajoute la sérialisation JSON à n'importe quelle classe."""
    def to_json(self) -> str:
        import json
        return json.dumps(self.__dict__, ensure_ascii=False, indent=2)

    @classmethod
    def from_json(cls, json_str: str):
        import json
        data = json.loads(json_str)
        obj = cls.__new__(cls)
        obj.__dict__.update(data)
        return obj

class LogMixin:
    """Ajoute Le logging automatique des appels de méthodes."""
    def log(self, message: str):
        import datetime
        horodatage = datetime.datetime.now().strftime("%H:%M:%S")
        print(f"[{horodatage}] {self.__class__.__name__} | {message}")

class ValidationMixin:
    """Ajoute une validation générique des attributs."""
    def valider(self) -> bool:
        for attr, valeur in self.__dict__.items():
            if valeur is None:
                self.log(f"Attribut '{attr}' est None") if hasattr(self, 'log') else None
            return False
        return True
```

Combinaison de mixins : ordre important (gauche → droite dans Le MRO)

```

class Produit(JsonMixin, LogMixin, ValidationMixin):
    def __init__(self, nom: str, prix: float, stock: int):
        self.nom = nom
        self.prix = prix
        self.stock = stock

    def vendre(self, quantite: int):
        self.log(f"Vente de {quantite} × {self.nom}")
        self.stock -= quantite

class Client(JsonMixin, LogMixin):
    def __init__(self, nom: str, email: str):
        self.nom = nom
        self.email = email

# Utilisation
p = Produit("Clavier mécanique", 149.9, 25)
p.vendre(3)
print(p.to_json())

c = Client("Bechir", "bechir@mail.com")
c.log("Connexion au compte")
json_client = c.to_json()
print(json_client)

# Reconstituer depuis JSON
c2 = Client.from_json(json_client)
print(f"Reconstitué : {c2.nom} — {c2.email}")

# Vérification MRO
print("\nMRO Produit :", [cls.__name__ for cls in Produit.mro()])

```

- **Mixin** : classe légère qui apporte un **comportement transversal** (logging, sérialisation, validation) sans modéliser une entité métier
- **règles d'un bon Mixin** : pas de `__init__`, pas d'état propre, ne s'instancie jamais seul, nom suffixé Mixin par convention
- **ordre dans l'héritage multiple** : les Mixins se placent **avant** la classe principale — `class Produit(JsonMixin, LogMixin, ClassePrincipale)`
- Très utilisé dans Django (ex: `LoginRequiredMixin`), DRF, et tous les grands frameworks Python pour ajouter des comportements sans toucher à l'arbre d'héritage principal

□ **Résumé des points les plus souvent testés en entretien** : `self` (convention, pas mot-clé) -attributs de classe mutables (piège liste

partagée) ·super() + MRO ·name mangling ≠ vraie privacité ·
@property pythonique ·valeurs par défaut mutables ·isinstance
vs type ·__slots__ pour la mémoire ·ABC pour les contrats ·
__call__ ·__new__ pour le Singleton ·composition > héritage ·
context managers ·Mixins